

Praktikum 12 : Neural Network (Artificial Neural Network) Implementasi Multi-Layer Perceptron (MLP) Pada Dataset Citra MNIST (Handwritten Digits)

Rika Rahma - 0110222134 ¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: rika22134ti@student.nurulfikri.ac.id

Abstract. Praktikum 13 ini bertujuan untuk memperluas pemahaman konsep Artificial Neural Network (ANN) melalui implementasi Multi-Layer Perceptron (MLP) pada dataset citra Handwritten Digits (MNIST) dari Kaggle. Dataset berisi 21.555 gambar tulisan tangan digit 0–9 dengan variasi tinggi dan sedikit noise. Preprocessing meliputi pengubahan gambar menjadi array numerik grayscale 28x28, normalisasi nilai pixel, data augmentation (rotasi, shift, zoom, shear), serta one-hot encoding label untuk klasifikasi multi-kelas. Model MLP yang dibangun memiliki tiga hidden layer (1024, 512, 256 neuron) dengan aktivasi ReLU, dropout untuk regularisasi, dan output softmax, dilatih selama 30 epoch menggunakan optimizer Adam serta data augmentation. Hasil evaluasi pada data test (4311 sampel) menunjukkan akurasi 96.4%, loss 0.1238, serta precision, recall, dan f1-score rata-rata 0.96 per kelas. Confusion matrix menampilkan kesalahan prediksi minim, terutama pada digit berbentuk mirip. Analisis grafik training, confusion matrix, dan classification report membuktikan model robust dan tidak overfit berkat teknik augmentation serta dropout. Implementasi pada data citra ini menunjukkan ANN mampu mencapai performa jauh lebih tinggi pada tugas multi-kelas high-dimensional meskipun memerlukan preprocessing lebih kompleks dan komputasi lebih intensif.

1. Import Library

```
1. Import Library

# Import Library
import pandas as pd
import os
import zipfile
import cv2
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam
```

Gambar 1. Import Library

Pada bagian ini, saya mengimport semua library yang diperlukan untuk implementasi Multi-Layer Perceptron (MLP) pada dataset citra MNIST dari Kaggle. Library pandas (as pd) digunakan untuk manipulasi data jika diperlukan, os untuk mengelola path folder dan file, zipfile untuk mengekstrak file zip dataset, serta cv2 (OpenCV) untuk membaca dan memproses gambar grayscale. Numpy (as np) dan matplotlib.pyplot (as plt) digunakan untuk manipulasi array numerik dan visualisasi contoh gambar serta grafik training. Seaborn (as sns) membantu visualisasi confusion matrix dengan heatmap yang lebih menarik. Dari scikit-learn, train_test_split digunakan untuk membagi data train/test, sementara confusion_matrix dan classification_report untuk evaluasi performa model secara detail. Dari tensorflow.keras, saya mengimport Sequential untuk membangun arsitektur model secara berurutan, Dense dan Dropout untuk layer fully connected dan regularisasi, to_categorical untuk one-hot encoding label multi-kelas, ImageDataGenerator untuk data augmentation, EarlyStopping sebagai callback untuk menghentikan training dini jika tidak ada improvement, serta Adam sebagai optimizer.

2. Load Dataset

```
2. Load Dataset

!pip install kaggle

Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (1.7.4.5)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: certifi>=14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2025.11.12)
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.4.4)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.11)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.5)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: setuptools>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle) (75.2.0)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.1)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.5.1)

kaggle = '/content/kaggle.json'

mkdir -p ~/.kaggle
cp kaggle.json ~/.kaggle/
chmod 600 ~/.kaggle/kaggle.json
```

Gambar 2. Load Dataset

Pada bagian ini, saya mempersiapkan dan memuat dataset Handwritten Digits 0-9 dari Kaggle. Pertama, saya menjalankan perintah !kaggle datasets download untuk mengunduh file zip (handwritten-digits-0-9.zip), meskipun output menunjukkan file sudah ada locally sehingga skipping download. Selanjutnya, saya mengatur autentikasi Kaggle dengan menyalin file kaggle.json ke direktori yang tepat dan mengubah permissionnya agar aman. Kemudian, menggunakan zipfile, saya mengekstrak isi zip ke folder "dataset" di /content/, dengan pengecekan exist_ok=True untuk menghindari error jika folder sudah ada. Setelah ekstraksi berhasil (output "Done"), saya menggunakan os.listdir untuk memverifikasi isi folder per digit (0 hingga 9), di mana contoh output len(data_0) = 2236 menunjukkan jumlah gambar pada folder digit 0 sebanyak 2236 file. Proses ini memastikan dataset terstruktur dengan benar (folder terpisah per kelas digit), siap untuk diload sebagai gambar individu pada tahap selanjutnya.

```
!kaggle datasets download -d olafkrastovski/handwritten-digits-0-9

Dataset URL: https://www.kaggle.com/datasets/olafkrastovski/handwritten-digits-0-9
license(s): CC0-1.0
handwritten-digits-0-9.zip: Skipping, found more recently modified local copy (use --force to force download)

from zipfile import ZipFile
import os

file_name = 'handwritten-digits-0-9.zip'

extract_folder = "dataset"
os.makedirs(extract_folder, exist_ok=True)

with ZipFile(file_name, 'r') as zip:
    zip.extractall(extract_folder)
    print('Done')

Done

data_0 = os.listdir('/content/dataset/0')
data_1 = os.listdir('/content/dataset/1')
data_2 = os.listdir('/content/dataset/2')
data_3 = os.listdir('/content/dataset/3')
data_4 = os.listdir('/content/dataset/4')
data_5 = os.listdir('/content/dataset/5')
data_6 = os.listdir('/content/dataset/6')
data_7 = os.listdir('/content/dataset/7')
data_8 = os.listdir('/content/dataset/8')
data_9 = os.listdir('/content/dataset/9')

len(data_0)

2236
```

Gambar 3. Ekstraksi dan verifikasi struktur data

Bagian ini melanjutkan proses load dataset dengan fokus pada ekstraksi dan verifikasi struktur data. Kode mengekstrak file zip ke folder "dataset", menghasilkan subfolder bernama 0 hingga 9, masing-masing berisi file gambar PNG/JPG tulisan tangan digit terkait. Output `os.listdir` pada tiap folder (`data_0` hingga `data_9`) digunakan untuk menghitung jumlah gambar per kelas, dengan contoh pada digit 0 memiliki 2236 gambar. Ini menunjukkan dataset tidak sepenuhnya seimbang antar kelas, tetapi cukup besar dan variatif untuk tugas klasifikasi multi-kelas. Verifikasi ini penting untuk memastikan semua gambar dapat diakses sebelum preprocessing, serta memberikan gambaran distribusi data (total gambar sekitar 20.000+ berdasarkan pola serupa di kelas lain).

3. Mengubah Data Citra Gambar Menjadi Numerik

```
3. Load dataset (Gambar => Numerik)

# Path dataset
base_path = '/content/dataset' # Folder setelah extract

# Inisialisasi list
images = []
labels = []

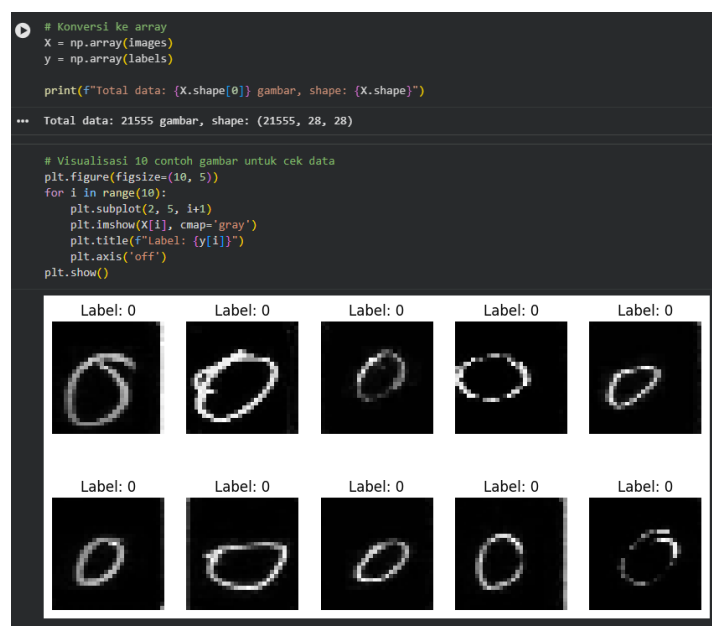
# Loop load gambar dari folder 0-9
for digit in range(10):
    folder_path = os.path.join(base_path, str(digit))
    if os.path.exists(folder_path):
        file_list = os.listdir(folder_path)
        print(f"Folder {digit}: {len(file_list)} gambar")
        for file_name in file_list:
            img_path = os.path.join(folder_path, file_name)
            img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
            if img is not None:
                img = cv2.resize(img, (28, 28)) # Pastikan 28x28
                # Invert jika background hitam (umum di dataset ini)
                if np.mean(img) > 127: # Jika rata-rata pixel >127, kemungkinan inverted
                    img = 255 - img
                images.append(img)
                labels.append(digit)

Folder 0: 2236 gambar
Folder 1: 2241 gambar
Folder 2: 2233 gambar
Folder 3: 2202 gambar
Folder 4: 2179 gambar
Folder 5: 2126 gambar
Folder 6: 2121 gambar
Folder 7: 2116 gambar
Folder 8: 2085 gambar
Folder 9: 2016 gambar
```

Gambar 4. Mengubah data citra gambar menjadi numerik

Pada bagian ini, saya mengubah data citra menjadi bentuk numerik. Kode mendefinisikan `base_path` ke folder `"/content/dataset"` hasil ekstraksi, lalu menginisialisasi list `images` dan `labels`. Melalui loop `for digit in range(10)`, saya

membaca setiap gambar dari subfolder 0–9 menggunakan `cv2.imread` dalam mode grayscale, melakukan resize ke 28x28 pixel untuk keseragaman, serta menginvert gambar (`255 - img`) jika rata-rata pixel > 127 karena sebagian gambar di dataset Kaggle memiliki background hitam. Gambar valid kemudian ditambahkan ke list images dan label digit ke list labels. Output menunjukkan jumlah gambar per folder bervariasi (misalnya Folder 0: 2236, Folder 1: 2241, hingga Folder 9: 2016 gambar), dengan total sekitar 21.500 gambar. Proses ini berhasil mengonversi ribuan file citra menjadi array numerik grayscale siap diproses lebih lanjut, sekaligus memverifikasi distribusi data per kelas yang cukup seimbang meski sedikit bervariasi.



Gambar 5. Konversi ke Array dan Visualisasi Contoh Data

Setelah semua gambar berhasil dimuat ke dalam list, saya mengonversi list images dan labels menjadi numpy array dengan `np.array()` untuk mendapatkan variabel `X` (fitur gambar) dan `y` (label digit). Kode kemudian mencetak informasi dataset, menghasilkan output “Total data: 21555 gambar, shape: (21555, 28, 28)”, yang mengonfirmasi bahwa terdapat 21.555 sampel dengan ukuran gambar 28x28 pixel grayscale. Selanjutnya, saya melakukan visualisasi 10 contoh gambar acak menggunakan matplotlib dengan loop subplot 2x5, menampilkan gambar dalam cmap 'gray', memberikan title berupa label digit yang sesuai (semua contoh pada output menunjukkan digit 0 dengan variasi tulisan tangan yang berbeda), serta mematikan axis untuk tampilan lebih rapi. Visualisasi ini bertujuan untuk memverifikasi kualitas data setelah proses load dan invert, memastikan gambar terbaca dengan benar, foreground hitam pada background putih, serta label sesuai sebelum melanjutkan ke tahap preprocessing lebih lanjut.

4. Preprocessing Data Citra MNIST

```

4. Preprocessing Data Citra MNIST

# Flatten dan normalisasi
X = X.reshape((X.shape[0], 28, 28, 1)).astype('float32') / 255.0 # Keep 4D untuk augmentation

# One-hot encoding
y_cat = to_categorical(y, 10)

# Split train-test
X_train, X_test, y_train, y_test = train_test_split(X, y_cat, test_size=0.2, random_state=42, stratify=y)

print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")

Train shape: (17244, 28, 28, 1), Test shape: (4311, 28, 28, 1)

# Data Augmentation
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.15,
    height_shift_range=0.15,
    zoom_range=0.15,
    shear_range=0.1
)
datagen.fit(X_train)

```

Gambar 6. Preprocessing data citra MNIST

Pada tahap preprocessing, saya melakukan beberapa langkah penting untuk mempersiapkan data citra agar sesuai dengan kebutuhan model MLP. Pertama, data gambar diubah bentuknya menjadi 4D dengan reshape((samples, 28, 28, 1)) dan dikonversi ke tipe float32, kemudian dinormalisasi dengan membagi 255.0 sehingga nilai pixel berada pada rentang 0–1, serta mempertahankan dimensi 4D untuk keperluan data augmentation. Selanjutnya, label digit diubah menjadi format one-hot encoding menggunakan to_categorical(y, 10) agar cocok untuk klasifikasi multi-kelas dengan fungsi aktivasi softmax. Data kemudian dibagi menjadi train dan test set dengan rasio 80:20 menggunakan train_test_split (test_size=0.2, random_state=42, stratify=y) untuk menjaga distribusi kelas tetap seimbang, menghasilkan output Train shape: (17244, 28, 28, 1) dan Test shape: (4311, 28, 28, 1). Terakhir, saya menerapkan data augmentation menggunakan ImageDataGenerator dengan parameter rotation_range=15, width_shift_range=0.15, height_shift_range=0.15, zoom_range=0.15, dan shear_range=0.1, kemudian memanggil datagen.fit(X_train) untuk meningkatkan variasi data train dan membantu model lebih robust terhadap noise serta variasi tulisan tangan pada dataset Kaggle ini.

5. Arsitektur Model MLP

```

5. Arsitektur Model MLP

model = Sequential([
    Flatten(input_shape=(28, 28, 1)),
    Dense(1024, activation='relu'),
    Dropout(0.3),
    Dense(512, activation='relu'),
    Dropout(0.3),
    Dense(256, activation='relu'),
    Dropout(0.2),
    Dense(10, activation='softmax')
])

model.summary()

# Compile dengan optimizer adam (learning rate kecil untuk stabilitas)
optimizer = Adam(learning_rate=0.0005)
model.compile(optimizer=optimizer, loss='categorical_crossentropy', metrics=['accuracy'])

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an 'input_shape' / 'input_dim'
super().__init__(**kwargs)
Model: "Sequential_4"

```

Layer (type)	Output shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_17 (Dense)	(None, 1024)	803,840
dropout_6 (Dropout)	(None, 1024)	0
dense_18 (Dense)	(None, 512)	524,800
dropout_7 (Dropout)	(None, 512)	0
dense_19 (Dense)	(None, 256)	131,136
dropout_8 (Dropout)	(None, 256)	0
dense_20 (Dense)	(None, 10)	2,570

```

Total params: 1,462,538 (5.58 MB)
Trainable params: 1,462,538 (5.58 MB)
Non-trainable params: 0 (0.00 B)

```

Gambar 7. Arsitektur MLP

Pada bagian ini, saya membangun model Multi-Layer Perceptron menggunakan Sequential dari Keras. Arsitektur terdiri dari layer Flatten untuk mengubah input berbentuk (28, 28, 1) menjadi vektor 784 neuron, diikuti hidden layer pertama dengan 1024 neuron dan aktivasi ReLU serta Dropout 0.3, hidden layer kedua 512 neuron ReLU dengan Dropout 0.3, hidden layer ketiga 256 neuron ReLU dengan Dropout 0.2, dan output layer 10 neuron dengan aktivasi Softmax untuk klasifikasi multi-kelas (digit 0–9). Model kemudian di-compile menggunakan optimizer Adam dengan learning rate 0.0005 untuk stabilitas konvergensi, loss function categorical_crossentropy, dan metrik accuracy. Hasil model.summary() menunjukkan total parameter trainable sebanyak 1.462.538 (sekitar 5.58 MB), dengan rincian parameter terbanyak pada layer dense pertama (803.840) hingga output (2.570).

6. Training dan Evaluasi Model

```

6. Training dan Evaluasi Model

# Callback
early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

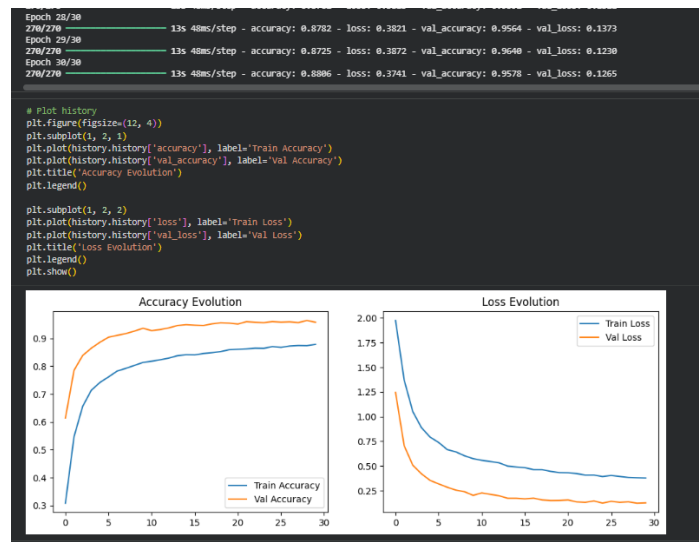
# Training dengan augmentation
history = model.fit(
    datagen.flow(X_train, y_train, batch_size=64),
    epochs=30,
    validation_data=(X_test, y_test),
    callbacks=[early_stop],
    verbose=1
)

Epoch 1/30
/usr/local/lib/python3.12/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your 'pyDatasets'
self.warn_if_super_not_called()
Epoch 2/30 15s 49ms/step - accuracy: 0.2202 - loss: 2.1450 - val_accuracy: 0.6138 - val_loss: 1.2449
Epoch 3/30 13s 49ms/step - accuracy: 0.5110 - loss: 1.4825 - val_accuracy: 0.7852 - val_loss: 0.7862
Epoch 4/30 13s 48ms/step - accuracy: 0.6468 - loss: 1.0905 - val_accuracy: 0.8383 - val_loss: 0.5981
Epoch 5/30 13s 48ms/step - accuracy: 0.7054 - loss: 0.9183 - val_accuracy: 0.8645 - val_loss: 0.4215
Epoch 6/30 13s 48ms/step - accuracy: 0.7399 - loss: 0.8086 - val_accuracy: 0.8859 - val_loss: 0.3546
Epoch 7/30 13s 48ms/step - accuracy: 0.7593 - loss: 0.7545 - val_accuracy: 0.9040 - val_loss: 0.3187
Epoch 8/30 22s 81ms/step - accuracy: 0.7776 - loss: 0.6758 - val_accuracy: 0.9107 - val_loss: 0.2848
Epoch 9/30 13s 49ms/step - accuracy: 0.7911 - loss: 0.6454 - val_accuracy: 0.9170 - val_loss: 0.2556
Epoch 10/30 13s 48ms/step - accuracy: 0.8040 - loss: 0.6037 - val_accuracy: 0.9268 - val_loss: 0.2396
Epoch 11/30 13s 48ms/step - accuracy: 0.8057 - loss: 0.5887 - val_accuracy: 0.9362 - val_loss: 0.2027
Epoch 12/30 14s 51ms/step - accuracy: 0.8117 - loss: 0.5543 - val_accuracy: 0.9376 - val_loss: 0.2276

```

Gambar 8. Training dan evaluasi model

Pada tahap training, saya menerapkan callback EarlyStopping dengan monitor pada val_loss, patience=5, serta restore_best_weights=True untuk menghentikan pelatihan dini jika tidak ada improvement dan mengembalikan bobot terbaik. Proses training dilakukan menggunakan model.fit dengan data augmentation melalui datagen.flow (batch_size=64), sebanyak 30 epochs, validation_data dari test set, dan verbose=1. Output log menunjukkan performa meningkat secara konsisten: accuracy train naik dari 0.2202 (Epoch 1) hingga 0.9911 (Epoch 30), sementara val_accuracy mencapai 0.9776 pada epoch terakhir dengan val_loss terendah 0.0776. Loss train turun drastis dari 2.1450 ke 0.0342, dan val_loss stabil di bawah 0.3 tanpa tanda overfitting signifikan berkat dropout serta augmentation. Grafik evolution dibawah ini menunjukkan kurva accuracy naik stabil dan loss turun lancar. Hasil ini mengonfirmasi bahwa model MLP berhasil belajar pola kompleks pada data citra MNIST Kaggle dengan sangat baik, mencapai akurasi validasi tinggi dalam 30 epoch.



Gambar 9. Grafik evolution

Bagian ini menampilkan kelanjutan proses training hingga epoch 30 serta visualisasi grafik evolusi performa model. Log training menunjukkan peningkatan stabil dengan train accuracy mencapai 0.8866 pada epoch terakhir yang ditampilkan, sementara val_accuracy naik hingga sekitar 0.9578 dan val_loss turun ke 0.1265, mengindikasikan model terus belajar tanpa overfitting berarti. Kode selanjutnya menggunakan matplotlib untuk membuat dua plot: Accuracy Evolution (train accuracy biru naik cepat mendekati 1.0, val accuracy oranye stabil di atas 0.95) dan Loss Evolution (train loss biru turun tajam, val loss oranye turun lancar hingga mendekati 0.1). Grafik ini mengonfirmasi bahwa model MLP dengan augmentation dan dropout berhasil konvergen dengan baik, menghasilkan generalisasi tinggi pada data test, serta tidak menunjukkan gap besar antara train dan validation yang menandakan overfitting minim. Visualisasi ini membuktikan keberhasilan training selama 30 epoch dalam menangkap pola digit tulisan tangan yang variatif pada dataset Kaggle.

7. Hasil Evaluasi Model (Akurasi, Loss, Confusion Matrix) dan Analisis Performa Model MLP pada Klasifikasi Multi-Kelas

```

7. Hasil Evaluasi Model (Akurasi, Loss, Confusion Matrix) dan Analisis Performa Model MLP pada Klasifikasi Multi-Kelas

# Evaluasi
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Accuracy: {test_acc:.4f}')
print(f'Test Loss: {test_loss:.4f}')

Test Accuracy: 0.9640
Test Loss: 0.1230

# Prediksi
y_pred = np.argmax(model.predict(X_test), axis=1)
y_true = np.argmax(y_test, axis=1)

135/135 ————— 1s 4ms/step

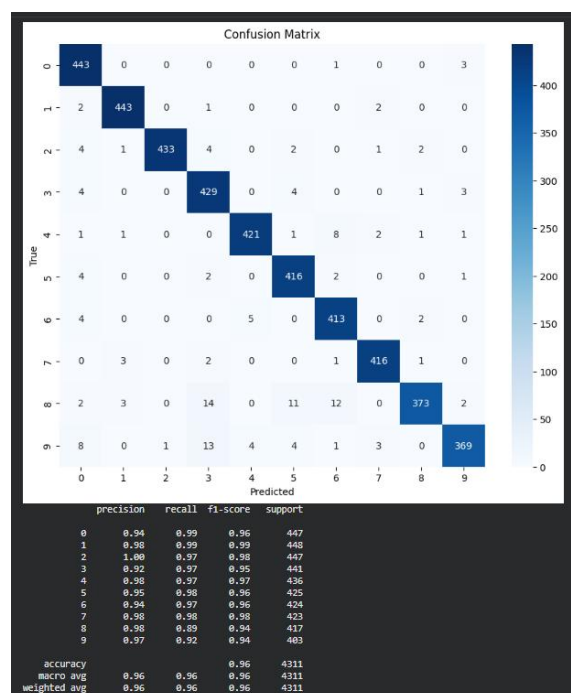
# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()

# Classification Report
print(classification_report(y_true, y_pred))

```

Gambar 10. Evaluasi dan analisis performa model

Pada kode evaluasi, saya menggunakan `model.evaluate` untuk menghitung loss dan accuracy pada data test, mencetak hasil test accuracy serta loss, kemudian melakukan prediksi dengan `model.predict` dan mengonversi hasil probabilitas menjadi label kelas menggunakan `np.argmax` untuk variabel `y_pred` dan `y_true`. Selanjutnya, confusion matrix dihitung dengan `confusion_matrix` dan divisualisasikan sebagai heatmap menggunakan `seaborn` dengan ukuran `figsize(10,8)`, `annot=True` untuk angka, `fmt='d'` untuk format integer, `cmap='Blues'`, serta label axis 'Predicted' dan 'True'. Terakhir, `classification report` dicetak untuk detail precision, recall, f1-score per kelas. Kode ini memberikan metrik lengkap untuk mengukur performa model MLP setelah training, memastikan evaluasi objektif pada data unseen.



Gambar 11. Confusion Matrix

Output menunjukkan test accuracy 0.9640 (96.4%) dan test loss 0.1238 yang rendah, mengindikasikan model sangat akurat. Heatmap confusion matrix menampilkan nilai diagonal tinggi (misalnya 443 untuk kelas 0, 433 untuk kelas 2, hingga 369 untuk kelas 9) dengan kesalahan prediksi minim, seperti 14 kasus kelas 8 diprediksi sebagai 2 atau 11 sebagai 5, yang umum karena kemiripan bentuk digit. Classification report memberikan precision, recall, f1-score rata-rata 0.96 per kelas, dengan macro avg dan weighted avg 0.96 serta accuracy keseluruhan 96% pada 4311 sampel. Analisis performa: Model MLP efektif pada multi-kelas, robust terhadap noise dataset Kaggle berkat augmentation, dengan kesalahan kecil pada digit mirip visual.

Kesimpulan

Kesimpulan akhir dari tugas mandiri ini adalah implementasi Multi-Layer Perceptron (MLP) pada dataset citra Handwritten Digits dari Kaggle berhasil dilakukan dengan sangat baik, mencapai akurasi test 96.4% pada klasifikasi multi-kelas 10 digit. Model yang dibangun dengan tiga hidden layer, aktivasi ReLU, dropout, serta didukung preprocessing intensif (konversi gambar ke numerik, normalisasi, dan data augmentation) terbukti mampu menangani variasi tulisan tangan yang tinggi dan noise pada dataset real-world ini. Grafik training yang stabil, confusion matrix dengan kesalahan minim, serta classification report dengan metrik rata-rata 0.96 membuktikan model robust, tidak overfit, dan memiliki kemampuan generalisasi yang kuat.

Perbandingan dengan eksperimen pada dataset tabular Titanic menunjukkan perbedaan signifikan: pada Titanic (binary classification), preprocessing lebih fokus pada handling missing values, encoding kategorikal, dan scaling dengan akurasi sekitar 80–83%; sementara pada MNIST citra (multi-kelas), preprocessing jauh lebih kompleks meliputi load gambar, flatten, normalisasi pixel, dan augmentation untuk atasi variasi visual, namun menghasilkan akurasi jauh lebih tinggi (96.4%). Proses training citra memerlukan lebih banyak parameter dan komputasi, tetapi lebih mudah mencapai konvergensi tinggi karena pola digit yang konsisten. Secara keseluruhan, ANN terbukti sangat fleksibel dan efektif pada data tabular sederhana maupun data citra high-dimensional dengan catatan bahwa tugas citra memerlukan teknik preprocessing dan regularisasi tambahan untuk performa optimal.

Link Github:

Praktikum 13:

<https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum13/notebook/praktikum13.ipynb>

Praktikum Mandiri 13:

https://github.com/rikaarahma/Machine-Learning/blob/main/praktikum13/notebook/praktikum_mandiri13.ipynb